TITLE OF THE PROJECT

BEACON

Final Year Project Report

by

Muhammad Samran Latif Khan
(051-23-138384)

Zain Ul Abideen
(051-23-138382)

In Partial Fulfillment

Of the Requirements for the degree

Bachelors of Science in Computer Science (BSCS)

Iqra University H-9 Campus

Islamabad, Pakistan

(2025)

FINAL APPROVAL

It is certified that the work printed in this report entitled **"BEACON"** by **Muhammad Samran Latif Khan,** registration No. 051-23-138384 and **Zain Ul Abideen** registration No. 051-23-138382 fulfill the requirements for award of degree of BS Computer Science from Department of Computing and Technology, Iqra University H-9 Campus, Islamabad, Pakistan

Viva Voce Committee

Supervisor

Internal Examiner-1: _____

Internal Examiner-2: _____

External Examiner:

DEDICATION

To our parents and families, whose patience and support carried us through every late night of debugging, retraining, and rewriting this project demanded. This work is as much a product of your quiet sacrifices as it is of our effort.

To Ms. Momina Maqbool, our supervisor, for the guidance that shaped this project at every stage, from its earliest, uncertain proposal to the design decisions that gave it real direction.

And to everyone who believes that a security tool is only as trustworthy as it is explainable, this project is dedicated to that principle.

DECLARATION OF ORIGINALITY

I, **Muhammad Samran Latif Khan,** registration No. 051-23-138384 and **Zain Ul Abideen** registration No. 051-23-138382 students of BS Computer Science, hereby declare that the work presented in the thesis entitled "**BEACON"** in partial fulfillment of BS degree in Computer Science from Iqra University, Islamabad, is my own work and has not been published or submitted as report research work or thesis in any form in any other university or institute in Pakistan or aboard.

Muhammad Samran Latif Khan (051-23-138384)

Zain Ul Abideen (051-23-138382)

Dated: 29th July, 2026

ACKNOWLEDGEMENT

I have no words to express our deepest gratitude to **Allah Almighty** for all of his countless blessings. I owe our deepest respect to HIS most noble messenger **Hazrat Muhammad (P.B.U.H),** who is forever, a torch of guidance and knowledge for all humanity. By virtue of his blessings today we are able to carry out our research work and present it.

The Almighty Allah has blessed me through the people who have contributed to the completion of this project. The first person who worth mentioning is my supervisor **Ma'am Momina Maqbool** who guided me and supported me during our whole development and research work. Without her help it was impossible to complete this hard task. She is an inspiring personality for me in all fields specially education. Under her supervision from the preliminary to the concluding level enabled me to develop an understanding of the field. Moreover, I would like to express our sincere thanks to all the faculty members of Department of Computing and Technology, IU Islamabad.

I also hereby extend my thanks to other lab fellows who made this project much easier for me with their guidance and suggestions. Of course, any my acknowledgment would never be completed without expressing gratitude to my parents. I owe everything to them and words are never enough express our attribution to my parents. My achievements in life would never complete without their prayers and support.

May Allah bless all these people

TABLE OF CONTENT

ABSTRACT

Malware continues to grow in scale and sophistication, and Pakistan's financial and government sectors have increasingly become direct targets of Remote Access Trojans and other intrusion tools. Existing malware detection systems generally suffer from two related shortcomings. Most produce a bare classification label with no supporting evidence, leaving a security analyst unable to judge whether a flagged process is genuinely malicious or a false alarm without manual investigation. In addition, many detection models are trained on a single data source, typically either network traffic or system memory, and so fail to capture the fuller behavioral picture that a real infection leaves behind.

This project, BEACON (Behavioral Explainable AI for Cyber Operations Network), addresses both problems together. The system is built around an XGBoost multiclass classifier trained on the BCCC Mal NetMemLog dataset, which provides network flow and memory forensic features captured from isolated executions of nine malware categories alongside benign samples. Rather than only assigning a label, every prediction is paired with a SHAP based explanation that surfaces the specific behavioral features that drove the classification, along with a qualitative risk rating intended to support rapid triage. Results are delivered through a Streamlit dashboard that accepts uploaded behavioral data and returns an instant, visually explained prediction.

Building this system required substantial groundwork before any model could be trained. The publicly available version of the dataset differs from the version originally described in the project's proposal, both in the number of malware categories represented and in how network and memory records are linked to one another, and this required a deliberate scope adjustment made in consultation with the project supervisor. A full preprocessing pipeline was then developed to clean the data, remove duplicate and inconsistent records, correct a labeling discrepancy between data sources, and address class imbalance, which is moderate on the network side and severe for one memory-based category with very few real-world samples. Established literature on imbalanced classification and gradient boosted models informed each of these decisions, and the resulting pipeline was validated end to end before proceeding to model training.

Compared to existing antivirus engines and commercial endpoint detection platforms, which largely operate as closed systems, and academic classifiers, which are typically evaluated on a single feature source, BEACON's contribution lies in combining a practical, validated multiclass classification pipeline with transparent, per prediction explanations aimed specifically at analyst usability. The current phase of the project has completed data acquisition, exploratory analysis, and preprocessing; model training, hyperparameter tuning, and evaluation follow directly from the pipeline described here. The expected outcome is a working prototype that demonstrates how explainable AI can be applied to real, imperfect security data without sacrificing the transparency that practical adoption depends on.

Chapter 1

INTRODUCTION

Every organization connected to the internet is, at some point, going to receive a file or a connection it did not ask for. Some of these are harmless. Others are ransomware, spyware, or remote access tools built specifically to sit quietly inside a network until they can do damage. Telling the two apart quickly, and correctly, is the job that malware detection software is supposed to do, and it is a job that has gotten harder rather than easier as attackers have grown more capable.

Most of the tools currently used to do this job share a common weakness. They are effective at flagging suspicious files, but they rarely explain why a file was flagged. A security analyst looking at an alert that simply reads "malicious, 94 percent confidence" has no way to judge whether that confidence is well placed without stopping to investigate the file manually, which takes time that is often in short supply during an active incident. This project grew out of that specific gap. It asks whether a detection system can be built that is not only accurate but also able to show its reasoning in terms a human analyst can immediately use.

BEACON, short for Behavioral Explainable AI for Cyber Operations Network, is the result. It is a system that looks at how a piece of software behaves, both on the network and inside a computer's memory, and classifies it into one of several malware families or as benign. Alongside that classification, it produces a short, human readable explanation naming the specific behavioral signals that led to the decision, together with a risk rating intended to help an analyst decide how urgently to respond. The whole system is wrapped in a simple web dashboard so that a security team can upload data and see results without needing to run any code themselves.

The people this project is built for are security operations center analysts, the staff responsible for triaging alerts in real time, though the underlying techniques are equally relevant to anyone building or studying machine learning systems for cybersecurity. For an analyst, the value is straightforward: less time spent second guessing an automated tool, and more confidence acting on what it reports.

This report documents the first phase of the project's development. Chapter Two sets out the problem being solved in more technical terms, explaining exactly what is missing from current approaches and why. Chapter Three reviews the published research that informed the project's design choices, particularly around handling imbalanced data and choosing an appropriate model, and situates BEACON relative to existing detection tools. Chapter Four specifies the software requirements for the system in detail, covering its intended functions, constraints, and the interfaces through which it will be used. Together, these chapters lay the groundwork for the model training and evaluation work that follows in the next phase of the project.

Chapter 2

PROBLEM DEFINITION

The general idea introduced in the previous chapter, that malware detection tools are accurate but unexplainable, needs to be broken down into something a developer can actually build against. Two separate problems are bundled together in that general complaint, and they call for different solutions.

The first is a transparency problem. Modern malware classifiers, including the gradient boosted and neural network models that now dominate the field, achieve strong accuracy precisely because they can model complex, nonlinear relationships between dozens or hundreds of behavioral features. That same complexity is what makes their internal reasoning opaque to a human reader. When such a model flags a process as ransomware, the label alone gives an analyst nothing to act on beyond trust in the model itself, and trust is exactly what is hardest to establish for a system whose decisions cannot be inspected. In a live incident, where the cost of a wrong response is high in either direction, an analyst is often forced to set the tool's output aside and re verify the finding manually, which defeats much of the point of automating detection in the first place.

The second problem is a data coverage problem. A malware infection leaves evidence in more than one place at once. Its network traffic shows patterns such as unusual connection frequency or data volume, while its presence in system memory shows up as anomalous process behavior, injected code, or irregular handle and module usage. Many published detection systems are trained on only one of these sources, typically network flow data, because it is easier to collect at scale. A model built this way inevitably misses whatever signal only shows up in the other source, and its accuracy in a real deployment, where both kinds of evidence are usually available, is correspondingly limited by that blind spot.

These two problems compound each other in practice. A system that is already hard to trust becomes harder still to justify deploying when it is also known to be working from incomplete evidence. The result, documented repeatedly in both industry reporting and academic literature, is that automated detection tools are frequently underused or manually double checked in exactly the high-pressure situations where their speed would matter most.

Framed as a concrete engineering problem, this project needed to answer the following question: can a single system be built that classifies software behavior across multiple malware families using both network and memory derived evidence, achieves accuracy competitive with existing single source approaches, and produces a per prediction explanation specific enough that an analyst can verify it without re running the entire investigation from scratch.

Answering that question turned out to require solving a set of more mundane problems first, and these are worth stating plainly because they shaped much of the early project work. Real world

security datasets of the kind needed for this project are large, often exceeding what fits comfortably in a personal computer's memory, and they are rarely as clean or as consistently structured as a textbook example. Malware families are not equally represented in captured data, since some behaviors are simply harder or slower to generate in a lab environment than others, which means a model trained naively on such data will tend to perform far better on common categories than rare ones. And where a dataset draws on more than one source of evidence, such as network captures and memory dumps taken from the same experiment, there is no guarantee that the two sources can be reliably linked back to the same underlying sample, which directly affects whether a fused, single model approach is even feasible. Chapter Four addresses how these constraints shaped the system's actual scope and requirements.

Chapter 3

LITERATURE REVIEW

Malware classification through machine learning is not a new idea, and a substantial body of published work already exists on training models to distinguish malicious from benign software using behavioral features. What is less settled, and what this project's early research specifically targeted, is how to handle two practical problems that arise the moment such a model is trained on real, imperfect data rather than a curated benchmark: how to correct for the fact that some malware categories are far better represented in training data than others, and how to keep a model's predictions interpretable once its accuracy depends on genuinely complex feature interactions.

On the question of imbalanced data, Tang, Tang, Liu and Cui (2026) offer a directly relevant comparison. Working on a stroke risk prediction task, itself an imbalanced classification problem in a different domain, they trained and compared logistic regression, random forest, XGBoost, CatBoost, and a neural network, applying SMOTE oversampling strictly within the training fold of each cross validation split to avoid leaking information into evaluation. Their finding that ensemble and boosted tree models handled the imbalance more robustly than simpler linear models, combined with their use of SHAP TreeExplainer for interpretation, closely mirrors the modeling choices this project ultimately made. Hassani, Entezarian, Zaeimzadeh, Marvian and Komendantova (2025) reached a related conclusion working on large scale record linkage, a task involving millions of imbalanced record pairs. They found that tree-based ensemble classifiers, including XGBoost, retained strong recall even without any resampling applied, while weaker models such as logistic regression and naive Bayes depended much more heavily on oversampling to reach comparable performance. This suggested that a lighter, less invasive imbalance correction could be sufficient wherever the underlying imbalance is only moderate.

Siagian, Sipayung, Rikki and Marbun (2025) examined the specific pairing of SMOTE with XGBoost across two datasets of very different size and domain, and their result that this combination generalized well across scale was useful evidence that the same approach should transfer reasonably to a dataset the size of the one used in this project. Their paper also made a smaller but practically important point, that features should be scaled before SMOTE is applied, since SMOTE generates synthetic samples through distance-based interpolation between neighboring points, and features left on very different numeric scales distort that interpolation in ways that are easy to miss until a model's minority class performance turns out worse than expected.

Where the imbalance in a dataset is severe rather than moderate, a plain application of SMOTE becomes less appropriate, since a small number of real minority samples can end up dominated by a large volume of near identical synthetic points generated from too few source examples. Salehi and Khedmati (2024) proposed a cluster-based alternative, CSBBoost, in which the minority class is first clustered and then oversampled proportionally within each cluster rather than as one undifferentiated group, preserving whatever internal sub structure the class actually has. Tested against eight competing hybrid sampling algorithms across twenty benchmark datasets with imbalance ratios reaching well over one hundred to one,

their approach consistently performed better, and it directly informed how this project handled its own most severely imbalanced category. Finally, de Vargas, Aranda, Costa, Pereira and Barbosa (2023) conducted a systematic mapping study across thirty five papers on imbalanced data preprocessing, concluding that hybrid sampling strategies and ensemble or neural network models consistently outperformed pure oversampling, pure undersampling, and classical models such as logistic regression or k nearest neighbors, across almost every application domain surveyed. This broader pattern gave confidence that the model and sampling choices made for this project were not domain specific quirks but a reasonably general finding.

Table 1: Comparative Analysis of Existing Malware Detection Approaches

Approach	**Purpose**	**Application Area**	**Strengths**	**Weaknesses**
Signature and heuristic based antivirus	Detect known malware through signature matching	Consumer and enterprise endpoints	Fast, low resource, mature and widely deployed	Cannot detect novel or heavily obfuscated malware; no behavioral explanation
Commercial endpoint detection and response (EDR) platforms	Real time behavioral monitoring and alerting	Enterprise security operations	Continuous monitoring, vendor supported, integrates with SOC workflows	Proprietary scoring logic, limited interpretability, ongoing licensing cost
Academic single source ML classifiers	Classify malware using one behavioral data source, usually network flow	Research and lab evaluation	Strong accuracy on the source they are trained on; well documented methodology	Miss signal present only in the other data source; single dataset evaluation is common
XAI augmented detection research	Add interpretability, e.g. SHAP or LIME, to a detection model	Academic research	Restores some analyst trust; exposes contributing features	Typically binary malicious versus benign; rarely combines multiple behavioral sources

Approach	Purpose	Application Area	Strengths	Weaknesses
BEACON (this project)	Multiclass malware family classification with per prediction explanation	SOC analyst decision support (prototype stage)	Combines network and memory derived features; full SHAP explanation; multiclass output with risk rating	Current dataset lacks a verified sample level link between network and memory sources, limiting fusion depth

The comparison makes the same gap visible from a different angle. Signature based tools are fast and mature but blind to anything novel. Commercial endpoint platforms add real time behavioral monitoring but keep their scoring logic proprietary, which is a reasonable business decision but leaves the analyst in the same unexplained position described in Chapter Two. Academic single source classifiers close part of the explainability gap in some cases, but rarely combine more than one behavioral data source, and the XAI augmented research prototypes that do add interpretability tend to stop at binary malicious versus benign decisions rather than distinguishing between malware families, which is the more operationally useful output for an analyst deciding how to respond.

Table 2: Gap Analysis for BEACON

Area	Current State	Identified Gap	Proposed Contribution
Explainability	Detection tools output a label and confidence score with little supporting reasoning	Analysts cannot verify predictions without manual reinvestigation	Per prediction SHAP explanation naming the top contributing behavioral features
Data Source Coverage	Most classifiers rely on a single behavioral data source, usually network traffic	Malware behavior visible only in memory, such as code injection, is missed	Incorporation of both network flow and memory forensic features within one pipeline
Class Imbalance Handling	Many published models apply one blanket oversampling technique regardless of severity	Uniform treatment either under corrects severe imbalance or inflates data unnecessarily	Differentiated handling: class weighting for moderate imbalance, cluster based synthetic oversampling for the one

Area	Current State	Identified Gap	Proposed Contribution
			severely underrepresented category
Analyst Usability	Advanced detection output is often accessible only through code, logs, or specialist tooling	Non specialist analysts cannot easily interact with model output	A browser based dashboard accepting direct file upload with immediate visual results
Detection Scope	Many academic tools demonstrate binary malicious versus benign classification only	Real SOC workflows need to know what kind of threat is present, not just that one exists	Multiclass classification across nine distinct malware categories plus benign
Cost and Accessibility	Commercial platforms with comparable behavioral monitoring require ongoing licensing	Smaller organizations and academic settings may lack access to equivalent tooling	An open, locally run prototype built entirely on freely available tools and libraries

Taken together, the published literature justified two decisions early enough in the project to save considerable rework later. First, that XGBoost, already selected in the original project proposal, was a defensible choice specifically because of its documented robustness to class imbalance relative to simpler models, not simply because it is a popular algorithm. Second, that imbalance correction should not be applied uniformly across the dataset, since the degree of imbalance itself varies substantially between the network derived and memory derived portions of the data used in this project, a distinction that is discussed further in Chapter Four.

Chapter 4

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

This chapter presents the Software Requirements Specification for BEACON, following the general structure recommended by IEEE Standard 830 for Software Requirements Specifications. It describes what the system is intended to do, the constraints it operates under, and the interfaces through which it will be used, at the level of detail needed to guide the development and evaluation work carried out in the remainder of the project.

4.1 Introduction

This section provides an overview of the entire Software Requirements Specification (SRS) document, outlining its purpose, scope, and the conventions used within it.

4.1.1 Document Purpose

This document specifies the software requirements for BEACON, an explainable multiclass malware classification system developed as a final year project at Iqra University. Its purpose is to give the project team, the project supervisor, and any future reader evaluating the system a single, shared reference for what BEACON is meant to do and how it is meant to behave, reducing the chance that later design or implementation decisions drift from what was originally intended.

4.1.2 Product Scope

BEACON, in its current phase, is a prototype that classifies behavioral data captured from executed software into one of nine categories: Backdoor, Benign, Exploit, HackTool, Hoax, Rootkit, Trojan, Virus, and Worm. These categories reflect the taxonomy used by the BCCC Mal NetMemLog dataset that the project draws on, which differs from the fifteen category scheme originally outlined in the project proposal. This adjustment was made after direct investigation of the dataset actually available for download, and was confirmed with the project supervisor before preprocessing work began. The system trains separate classifiers on network flow features and on memory forensic features respectively, since the dataset does not currently provide a verified way to link records from the two sources back to the same underlying sample. Each classifier is paired with a SHAP based explanation layer, and results are surfaced through a Streamlit dashboard. Live network packet capture, deployment on production SOC infrastructure, and deep learning architectures are explicitly out of scope for this phase and are noted as potential future work.

4.1.3 Definitions, Acronyms, and Abbreviations

- BEACON: Behavioral Explainable AI for Cyber Operations Network, the system described in this document.
- SOC: Security Operations Center, the team responsible for monitoring and responding to security alerts.
- XAI: Explainable Artificial Intelligence, machine learning techniques designed to make model output interpretable to a human reader.
- SHAP: SHapley Additive exPlanations, the explainability technique used to attribute a prediction to individual input features.
- XGBoost: Extreme Gradient Boosting, the gradient boosted decision tree algorithm used as BEACON's core classifier.
- SMOTE: Synthetic Minority Oversampling Technique, a method for generating synthetic samples of an underrepresented class.
- BCCC: the dataset provider and category taxonomy this project's data is sourced from.
- EDA: Exploratory Data Analysis.
- SRS: Software Requirements Specification, the type of document this chapter presents.
- FYP: Final Year Project.

4.1.4 References

This section lists all documents and web resources referenced in this SRS. This includes relevant standards, existing system documentation, and any other materials that provide context or supplementary information.

- Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Referenced for the core algorithm underlying BEACON's classifiers. https://www.kdd.org/kdd2016/papers/files/rfp0697-chenAemb.pdf

- Lundberg, S. M., and Lee, S. I. (2017). A unified approach to interpreting model predictions. Referenced for the SHAP methodology used in the explainability layer. https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf

- de Vargas, V. W., Aranda, J. A. S., Costa, R. dos S., Pereira, P. R. da S., and Barbosa, J. L. V. (2022). Imbalanced data preprocessing techniques for machine learning: a systematic mapping study. Knowledge and Information Systems, 65, 31-57. https://link.springer.com/article/10.1007/s10115-022-01772-8
- Hassani, H., Entezarian, M. R., Zaeimzadeh, S., Marvian, L., and Komendantova, N. (2025). An oversampling-undersampling strategy for large-scale data linkage. Frontiers in Big Data, 8, 1542483. https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2025.1542483/full

- Salehi, A. R., and Khedmati, M. (2024). A cluster-based SMOTE both-sampling (CSBBoost) ensemble algorithm for classifying imbalanced data. Scientific Reports, 14, 5152. https://www.nature.com/articles/s41598-024-55598-1

- Siagian, N. A., Sipayung, S. P., Rikki, A., and Marbun, N. (2025). Integrating SMOTE with XGBoost for robust classification on imbalanced datasets: a dual domain evaluation. Sinkron: Jurnal dan Penelitian Teknik Informatika, 9(3), 1094-1107. https://jurnal.polgan.ac.id/index.php/sinkron/article/view/15029

- Tang, X., Tang, M., Liu, W., and Cui, S. (2026). Explainable machine learning for stroke risk prediction: a comparative study with SHAP based interpretation. Frontiers in Neurology, 16, 1716984. https://www.frontiersin.org/journals/neurology/articles/10.3389/fneur.2025.1716984/full

- BCCC Mal NetMemLog dataset documentation, Behaviour-Centric Cybersecurity Center, York University. https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/large-scale-multisources-malware-analysis-dataset-using-network-traffic-and-memory-bccc-mal-netmem-2025/

4.1.5 Document Overview

The remainder of this chapter is organized as follows. Section 4.2 gives an overall description of the product, covering its perspective relative to existing tools, its main functions, its constraints, its intended users, and the assumptions the project depends on. Section 4.3 specifies detailed requirements, both functional and non functional, along with the external interfaces through which the system is accessed. Section 4.4 describes how these requirements will be verified, and Section 4.5 notes the supplementary material available in the project's appendixes.

4.2 Overall Description

This section describes the general factors that affect the product and its requirements, providing a background for the detailed requirements defined in Section 4.3.

4.2.1 Product Perspective

BEACON is a new, standalone academic prototype rather than a replacement for any existing security product. It is intended to sit alongside, not instead of, conventional monitoring tools such as antivirus engines or commercial endpoint detection platforms, functioning as a decision support layer that a SOC analyst consults when an alert needs a second, more transparent opinion. The system consumes pre extracted behavioral feature data, either network flow statistics or memory forensic measurements, and does not itself perform packet capture or memory acquisition.

4.2.2 Product Functions

At a high level, BEACON is responsible for the following functions:

- Ingesting and validating uploaded behavioral feature data.
- Producing a multiclass prediction identifying the most likely malware category or benign status.
- Generating a SHAP based explanation identifying the features that most influenced that prediction.
- Assigning a qualitative risk level to help prioritize analyst attention.
- Presenting all of this through an interactive dashboard.

A parallel, offline set of functions, not exposed through the dashboard, handles dataset preprocessing, model training, and hyperparameter tuning, and is used by the development team rather than an end user.

4.2.3 Product Constraints

Several constraints shape what BEACON can and cannot do in its current form. The raw dataset exceeds one hundred gigabytes across all categories, which rules out loading it into memory on a typical personal computer all at once and requires chunked, file by file processing during preprocessing. Development hardware is limited to a personal laptop with eight gigabytes of RAM or more, with Google Colab available as a fallback for heavier training runs. The project timeline is bounded by the academic semester structure governing final year project submissions. Most significantly, the dataset does not currently provide a confirmed way to link a given network capture to the memory dump of the same executed sample, since the two are identified using entirely different naming conventions. This constrains the system to two coordinated but separately trained classifiers rather than a single model operating on fused features, at least until a reliable linkage becomes available.

4.2.4 User Characteristics

Two user classes are relevant to this system. The primary intended user is a SOC analyst, assumed to have working knowledge of malware categories and network security concepts but no particular background in machine learning, who needs the dashboard's output to be immediately actionable without further technical interpretation. The secondary user class consists of the project's own developers and, by extension, any future student or researcher extending the work, who interact with the system's training and evaluation scripts directly and are expected to have Python and machine learning experience.

4.2.5 Assumptions and Dependencies

The project assumes continued availability of the BCCC Mal NetMemLog dataset in its current form and category taxonomy for the remainder of the development timeline. It depends on a set of third party open source libraries, principally XGBoost, SHAP, Polars, pandas, scikit-learn, and Streamlit, remaining stable and compatible with one another across the Python version used for

development. It also assumes that the two stream architecture adopted in response to the dataset's linkage limitation remains an acceptable approach to the project supervisor, pending any future update to the dataset or further guidance on the matter.

4.2.6 Apportioning of Requirements

The core classification and explanation functions, along with the offline preprocessing and training pipeline, are treated as mandatory for the current project phase. Dashboard refinements beyond a functional minimum, a baseline comparison against a second model such as Random Forest, and a fully fused network and memory architecture, should a reliable sample level linkage become available, are treated as extensions that may be deferred to a later phase of the project or noted as future work if time does not permit their completion.

4.3 Specific Requirements

This section specifies all software requirements in detail, ensuring they are sufficient for design and testing. Each requirement should be uniquely identifiable, verifiable, and clearly stated.

4.3.1 External Interfaces

This subsection defines all inputs and outputs of the software system, including user, hardware, software, and communication interfaces.

4.3.1.1 User Interfaces

The system is accessed through a Streamlit based web dashboard running in a standard browser. The interface provides a file upload control for submitting a CSV file of behavioral features, a results panel showing the predicted category with a numeric confidence score, a color coded risk level indicator, and an embedded SHAP visualization showing the top contributing features for that specific prediction.

4.3.1.2 Hardware Interfaces

BEACON does not require any specialized hardware. It runs on standard consumer or laptop grade hardware for both training and inference, with optional access to cloud based compute such as Google Colab for training runs that exceed local resource limits.

4.3.1.2 Software Interfaces

The system depends on a Python 3.10 or later runtime, together with the XGBoost, SHAP, scikit-learn, Polars, pandas, and NumPy libraries for data processing and modeling, and the Streamlit framework for the user facing dashboard. It reads and writes data using standard CSV and Parquet file formats through the host operating system's file system.

4.3.1.3 Communications Interfaces

The dashboard is served over a local HTTP interface, by default accessible at localhost within the machine running it. No external network communication is required during inference, since the trained model and explanation logic run entirely on the local machine against locally uploaded data.

4.3.2 Functional Requirements

FR-1: The system shall accept a CSV file containing network flow or memory forensic feature data as input through the dashboard.

FR-2: The system shall validate an uploaded file against the expected feature schema and reject files that do not match before attempting a prediction.

FR-3: The system shall classify a valid input record into one of the nine supported malware categories or as benign.

FR-4: The system shall report a numeric confidence score alongside the predicted category.

FR-5: The system shall compute SHAP values for each prediction and display the top contributing features in a readable visual format.

FR-6: The system shall assign a qualitative risk level of Critical, High, Medium, or Low based on the predicted category and its associated confidence.

FR-7: The system shall allow the user to export or download the prediction result and its accompanying explanation.

FR-8: The development pipeline shall support retraining and reevaluation of the underlying models independently of the dashboard, for use by the project team during development.

4.3.3 Quality of Service Requirements (Non-Functional Requirements)

This section specifies the non-functional requirements, which are constraints on the system's operations and attributes. These include performance, security, reliability, availability, usability, maintainability, portability, scalability, interoperability, and compliance.

4.3.3.1 Performance

A single uploaded record should return a prediction and its SHAP explanation within a few seconds under normal operation on standard hardware. Full dataset preprocessing and model training are

understood to be considerably longer running processes and are performed offline, separately from the live dashboard.

4.3.3.2 Security

Uploaded data is processed locally and is not transmitted to any external server. No uploaded sample is retained beyond the active session unless the user explicitly chooses to save a result.

4.3.3.3 Reliability

The system should handle malformed, incomplete, or unexpectedly formatted input gracefully, returning a clear validation message rather than failing without explanation, consistent with the validation and imputation logic already established in the preprocessing pipeline.

4.3.3.4 Availability

As an academic prototype, BEACON is available whenever its dashboard process is actively running on the host machine, with no formal uptime guarantee expected at this stage. Continuous availability requirements would need to be revisited separately if the system were ever adapted for production SOC deployment, which remains outside the current project scope.

4.3.4 Compliance

The project follows the documentation and evaluation requirements set by the Department of Software Engineering at Iqra University for final year projects. Use of the BCCC Mal NetMemLog dataset is restricted to the academic research purposes for which it was obtained, and the dataset itself consists of lab generated, isolated malware executions rather than real world captured traffic, so no personally identifiable network data is involved.

4.3.5 Design and Implementation Constraints

The system is implemented in Python 3.10 or later. XGBoost is the mandated core classification algorithm, consistent with the original project proposal, and SHAP's TreeExplainer is used for interpretability given its efficiency on tree based models. Streamlit is the required framework for the dashboard. Development uses Visual Studio Code with GitHub for version control and Trello for task tracking. The two stream, non fused architecture described in Section 4.2.3 is treated as a current design constraint rather than a permanent one, since it follows directly from a present data limitation rather than a deliberate architectural preference.

4.4 Verification

Verification of BEACON's requirements is carried out at two levels. At the data level, the preprocessing pipeline is checked directly after each stage for issues such as remaining missing values, unexpected value ranges, and consistency between training and test partitions, using a dedicated validation script run before any model is trained on the output. At the model level, performance is assessed on a held out test set produced through a grouped, stratified split designed to prevent related records from appearing in both training and testing, using standard classification metrics including per class precision, recall, F1 score, and confusion matrices, with particular attention paid to categories known to be underrepresented in the source data. SHAP output is additionally reviewed manually against domain expectations for each malware category, to check that the features a prediction is attributed to are behaviorally plausible rather than artifacts of the data collection process. The project supervisor reviews this verification evidence as part of the standard project evaluation process.

4.5 Appendixes

Supplementary material supporting this specification, including the full list of features retained after correlation based pruning, the category wise class distribution of the dataset before and after preprocessing, and the preprocessing scripts referenced throughout this document, is maintained alongside the project's source code repository rather than reproduced in full here.

Chapter 5

Software Design Specification

Malware classification systems built on machine learning are typically evaluated on accuracy alone, which hides two problems that matter directly to a security analyst using the system: first, most public malware telemetry is severely class-imbalanced, so a model can score well overall while essentially failing to detect rare-but-critical categories; second, even a well-performing model gives the analyst no way to see why a sample was flagged, which limits how much the classification can be trusted or acted on. This is the specific, scoped problem Beacon addresses: it classifies malware samples represented as network-flow telemetry and, independently, as memory-dump telemetry, across 9 categories (Backdoor, Benign, Exploit, HackTool, Hoax, Rootkit, Trojan, Virus, Worm) drawn from the BCCC-Mal-NetMem-2025 dataset, while explicitly correcting for the imbalance present in that data and surfacing a SHAP-based explanation alongside every prediction. If left unsolved, an analyst using an uncorrected, unexplained classifier risks two concrete failures: missing rare malware categories (e.g., the severely under-represented Exploit class in the memory-dump data) and trusting predictions that cannot be independently verified against known malicious behavior.

5.1 Design Methodology and Software Process Model

Design Methodology: Object-Oriented

Beacon follows an object-oriented design methodology rather than a purely procedural one. This choice is justified by three characteristics of the system itself:

- **Encapsulation of pipeline stages:** cleaning, missing-value handling, scaling, resampling, model training, and explanation generation are each distinct responsibilities with their own internal state (e.g. fitted Min-Max ranges, a trained model's feature order) - modeling each as a class keeps that state bundled with the logic that uses it, instead of passing loose parameters between functions.
- **Reuse through inheritance and polymorphism:** the Network and Memory pipelines are structurally identical (same 8 preprocessing steps) but differ in specifics (imbalance ratio, resampling strategy). A common MalwareClassifier base class with NetworkClassifier/MemoryClassifier subclasses, and a common Resampler base class with SmoteResampler/ClusterBasedResampler subclasses, let the shared logic live in one place while each subclass overrides only what differs (Section 5.4, Class Diagram).
- **Consistency with the underlying libraries:** XGBoost, scikit-learn, and SHAP all expose object-oriented APIs (fit/predict/transform-style objects). Wrapping Beacon's own components the same way keeps the codebase consistent and easier to extend - e.g. adding a LightGBM-based classifier later is a new subclass, not a rewrite.

Software Process Model: Iterative and Incremental

Beacon follows an iterative and incremental process model rather than a strict waterfall model. This was a necessity rather than a preference, for a concrete reason surfaced during the project itself: the exploratory data analysis (EDA) performed after the initial design assumptions were set uncovered findings that required revisiting earlier decisions - most notably, that the Network and Memory feature spaces do not merge on sample_id (making a single fused model impossible, contrary to the original single-model assumption), that the Memory-side Exploit class is severely imbalanced (requiring a heavier cluster-based resampling technique not needed elsewhere), and a data-reporting anomaly in the Rootkit category that had to be verified before training could proceed.

A waterfall model, which assumes requirements and design are frozen before implementation begins, would not have accommodated this - the correct architecture (two independent classifiers) and the correct resampling strategy per dataset were only knowable after the data itself was analyzed. The project was therefore organized into short, checkpointed iterations - proposal, EDA, preprocessing-and-modeling design (this document), implementation, and evaluation - each reviewed with the project supervisor before the next iteration began, with each iteration allowed to feed corrections back into the design.

5.2 System Overview

Beacon is an explainable malware classification system that ingests a malware sample's telemetry - either network-flow records or memory-dump records - and returns both a predicted category (one of 9: Backdoor, Benign, Exploit, HackTool, Hoax, Rootkit, Trojan, Virus, Worm) and a SHAP-based explanation of that prediction, through a Streamlit web dashboard.

Background: the system is built on the BCCC-Mal-NetMem-2025 dataset (York University), which profiles each malware sample from two angles - a Network-flow capture (348 features: byte/packet rates, durations, port behavior, etc.) and a Memory-dump capture (98 features: process-memory structures). Analysis of the dataset found that these two feature spaces do not correspond row-for-row (0% match rate joining on sample_id across all 9 categories), so Beacon is designed as two independent, parallel classification pipelines rather than one fused model - a Network Detection pipeline and a Memory Detection pipeline, sharing the same preprocessing approach and explanation mechanism but trained, resampled, and evaluated separately (Section 5.2.1).

Both pipelines share the same top-level flow: a raw CSV upload is cleaned and transformed by the Preprocessing module (Steps 1-8, applied identically at training time and inference time so the dashboard never processes data differently than the model was trained on), classified by a trained XGBoost model, and explained via a SHAP TreeExplainer - with the result rendered back to the analyst through the dashboard's Network Detection, Memory Detection, or About/Methodology page.

5.2.1 Architectural Design

Beacon decomposes into eight modules, grouped functionally as follows:

1. **Data Ingestion Module:** accepts an uploaded CSV (network-flow or memory-dump records) and hands it to Preprocessing.

2. **Preprocessing Module:** Steps 1-8 (duplicate removal, empty-column drop, label normalization, missingness handling, log-transform, Min-Max scaling, correlation pruning); identical code path used at training and inference.

3. **Resampling Module:** light multiclass SMOTE for the Network model (moderate ~5:1 imbalance); cluster-based oversampling (CSBBoost-style) for the Memory model's Exploit class (severe ~10-16:1 imbalance). Used only during training, not at inference.

4. **Model Training Module:** offline scripts that orchestrate Preprocessing + Resampling + XGBoost training, and persist the result to the Artifact Store.

5. **Inference/Classification Module:** loads a persisted model and its preprocessing artifacts, and produces predictions and class probabilities for new data.

6. **Explainability Module:** wraps SHAP's TreeExplainer to produce per-prediction feature attributions.

7. **Presentation Module:** the Streamlit multi-page dashboard (landing page, Network Detection, Memory Detection, About/Methodology) that an analyst interacts with directly.

8. **Artifact Store:** persists trained models and their associated preprocessing artifacts (Min-Max ranges, dropped-column lists, feature order) so training and inference always agree on how data was transformed.

The key relationship to note is that the Preprocessing Module is shared, not duplicated, between the Model Training Module and the Inference/Classification Module - the same functions in pipeline/common.py are called by both, with training fitting the transformation parameters (Min-Max ranges, correlation drop-list) and inference simply applying them. This is what guarantees the dashboard cannot silently drift from how the model was actually trained.

Initial Design: Box-and-Line Diagram

The diagram below shows the initial, simplified view of how data moves through the system, before an architecture style was finalized.

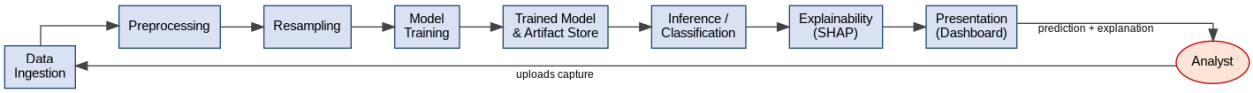

Figure 5.1 - Box-and-line diagram of Beacon's module flow.

Finalized Architecture Style: Layered Architecture

Beacon's finalized architecture follows a Layered Architecture style, chosen because the system has a natural, one-directional dependency chain - the dashboard (presentation) depends on preprocessing and explanation services (application/service layer), which depend on trained models (model/intelligence layer), which depend on stored data and artifacts (data layer) - with no layer depending upward. This is a simpler fit than a full client-server or microservice style, since Beacon currently runs as a single Streamlit process rather than communicating over a network between separately deployed components; a layered style still leaves room to later split the Model/Intelligence layer behind a FastAPI service boundary (noted as a future extension) without restructuring the layers themselves.

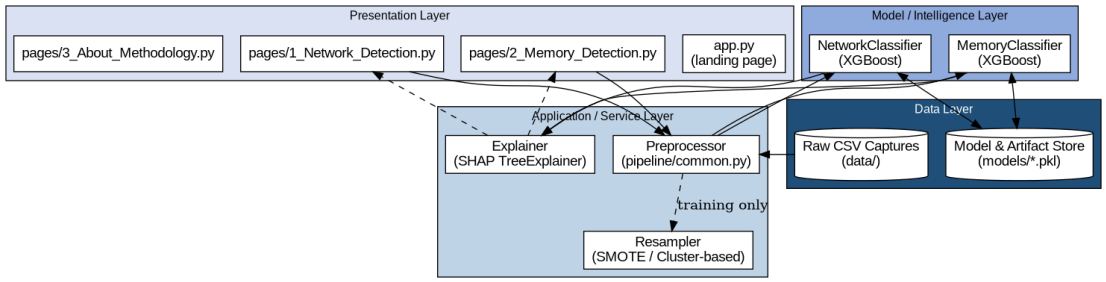

Figure 5.2 - Layered architecture with modules mapped to each layer.

Module-to-Layer Mapping

Layer	Modules / Components	Responsibility
Presentation	app.py; pages/1_Network_Detection.py; pages/2_Memory_Detection.py; pages/3_About_Methodology.py	File upload, results display, SHAP chart rendering, navigation
Application / Service	Preprocessor (pipeline/common.py); Resampler (SmoteResampler, ClusterBasedResampler); Explainer	Data cleaning/transformation shared by training and inference; class-imbalance correction; SHAP computation
Model / Intelligence	NetworkClassifier; MemoryClassifier (both XGBoost-based)	Learned mapping from preprocessed features to one of 9 malware categories
Data	Raw CSV captures (data/); Model & Artifact Store (models/*.pkl)	Persistent storage of input data, trained models, and preprocessing artifacts

5.4 Design Models

Beacon follows the object-oriented development approach (Section 5.1), so the applicable design models are a Class Diagram, Sequence Diagram, State Transition Diagram, Activity Diagram, and Data Flow Diagram, presented below. All diagrams follow UML 2.5 notation conventions (hollow-triangle arrowheads for inheritance, open-diamond for aggregation/uses relationships, solid arrows for associations and message calls).

5.4.1 Class Diagram

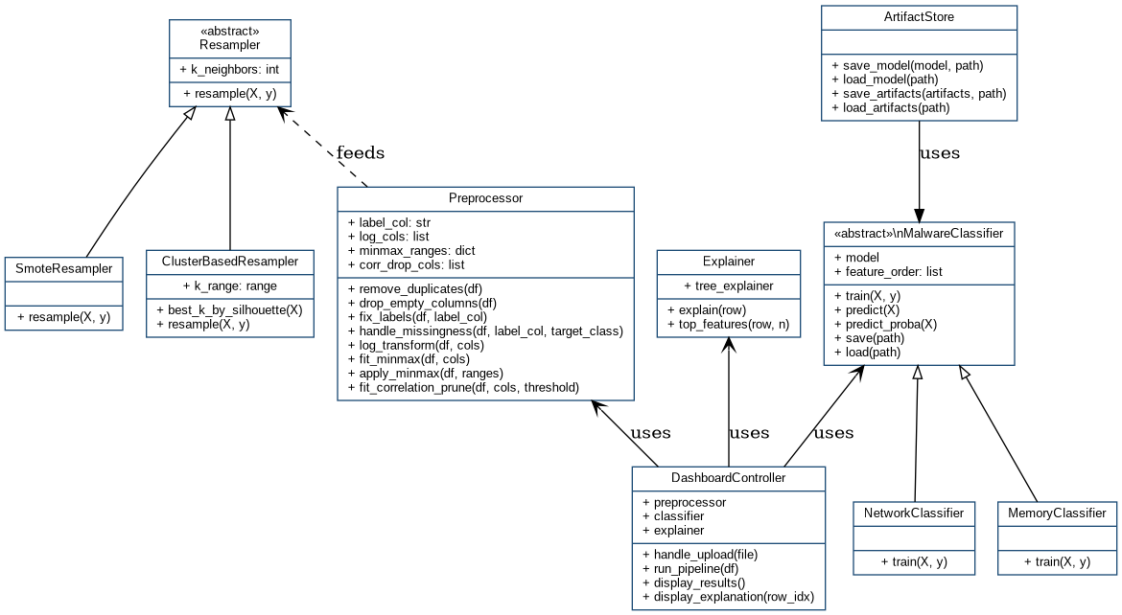

Figure 5.3 - Class diagram of Beacon's core components.

Preprocessor centralizes Steps 1-8 and is used by both the training scripts and, at inference time, by DashboardController (via apply_preprocessing). Resampler is an abstract base with two concrete strategies - SmoteResampler for the moderately imbalanced Network data, and ClusterBasedResampler (CSBBoost-style: K-means clustering via best_k_by_silhouette, then per-cluster SMOTE) for the severely imbalanced Memory Exploit class - selected polymorphically depending on which dataset is being trained. MalwareClassifier is an abstract base wrapping a trained XGBoost model, with NetworkClassifier and MemoryClassifier as concrete subclasses that differ only in which data and resampling strategy trained them, not in their prediction interface. Explainer wraps a SHAP TreeExplainer and is shared by both classifiers. DashboardController is the Presentation-layer object that orchestrates a single request: receiving an upload, calling Preprocessor, calling the appropriate MalwareClassifier, calling Explainer, and returning results for display. ArtifactStore is used by MalwareClassifier to persist and reload both the trained model and its associated preprocessing parameters as one bundle.

5.4.2 Sequence Diagram

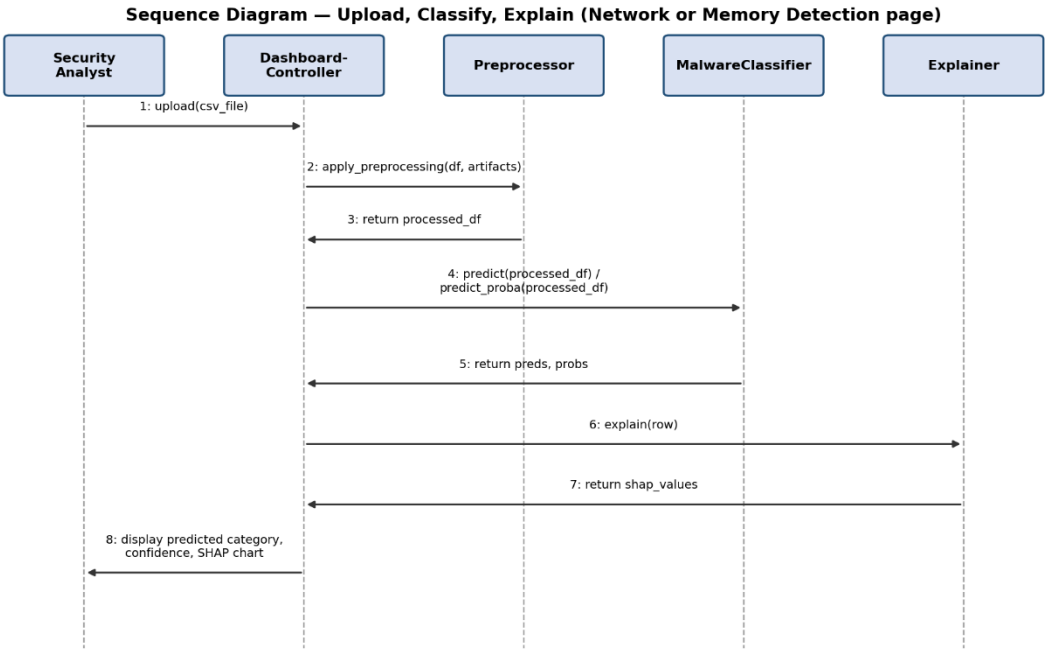

Figure 5.4 - Sequence diagram for a single upload-classify-explain request.

This sequence corresponds to a single analyst interaction on either the Network Detection or Memory Detection page. DashboardController first delegates to Preprocessor to bring the uploaded data into the exact numeric form the model expects (reusing the fitted Min-Max ranges and correlation drop-list from training), then to MalwareClassifier for the predicted category and class probabilities, then to Explainer for a per-row SHAP attribution, before rendering all three results back to the analyst in one view.

5.4.3 State Transition Diagram

Figure 5.5 - State transitions for a single uploaded record.

A record moves through five states from upload to display. An uploaded record that fails schema validation (e.g. missing required columns) transitions to Rejected rather than proceeding through the pipeline, so malformed input is caught before it can reach the model.

5.4.4 Activity Diagram

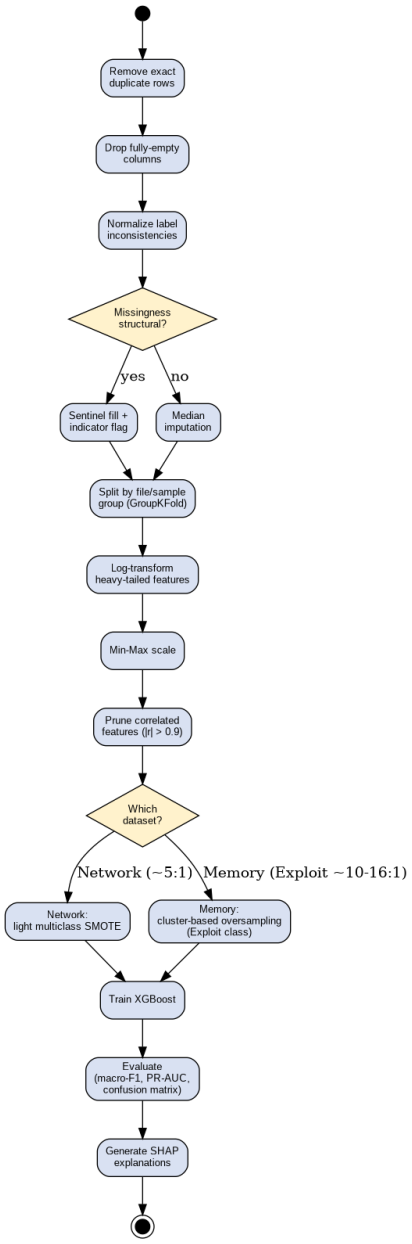

Figure 5.6 - Activity diagram of the full preprocessing-through-explanation pipeline.

This diagram reflects the full Steps 1-13 pipeline described in the project's preprocessing guide, including its two decision points: whether a missing-value pattern is structural or random (Step 4, determining sentinel+indicator vs. median fill), and which dataset is being processed (Step 9/10, determining light SMOTE vs. cluster-based oversampling).

5.4.5 Data Flow Diagrams

Level 0 (Context Diagram)

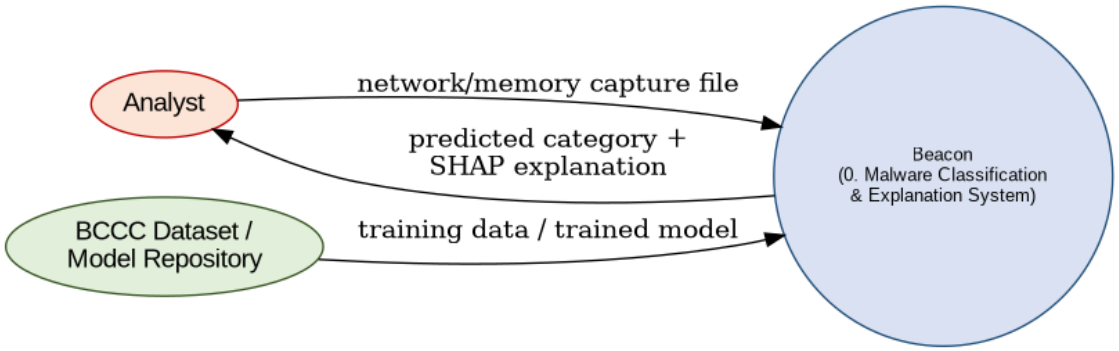

Figure 5.7 - DFD Level 0, showing Beacon as a single process bounded by its external entities.

Level 1

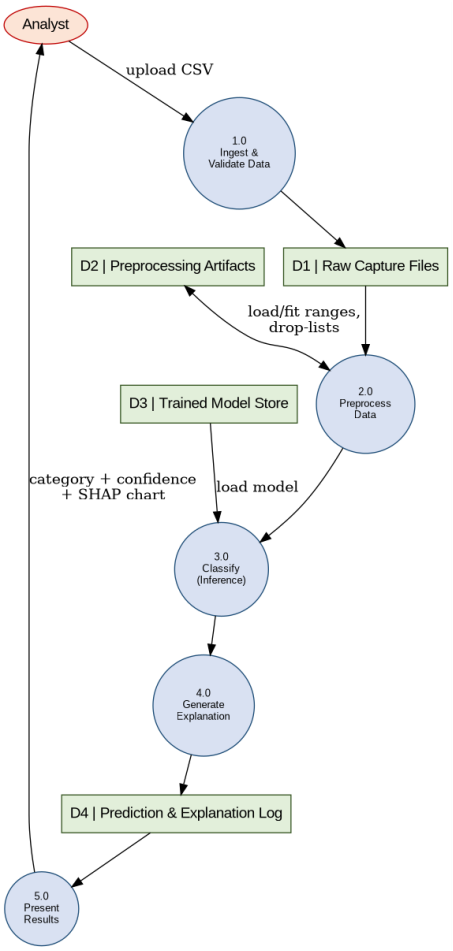

Figure 5.8 - DFD Level 1, decomposing Beacon into its five major processes and four data stores.

D1 (Raw Capture Files) and D3 (Trained Model Store) are read-only from the perspective of a live classification request; D3 and D2 (Preprocessing Artifacts) are written once, offline, by the Model

Training Module (Section 5.2.1), and only read during inference. D4 (Prediction & Explanation Log) is the extension point noted in Section 5.5 for the planned move from in-session-only results to persisted history.

5.5 Data Design

Beacon's information domain starts as heterogeneous CSV rows (numeric flow/memory statistics, a small number of categorical fields, and a category label) and is transformed by the Preprocessing Module into fixed-length numeric feature vectors suitable for XGBoost - duplicate and empty-column removal reduce row/column noise, label normalization guarantees exactly 9 clean category values, log-transform and Min-Max scaling normalize each feature's range, and correlation pruning removes redundant columns before the vector reaches the model. The resulting entities - feature vectors, predictions, and SHAP explanations - are represented in-memory as pandas DataFrames/NumPy arrays, and persisted to disk as pickled artifacts (via joblib) rather than in a relational database in the current implementation; prediction results are currently shown only within a Streamlit session and are not yet logged persistently (D4 in Figure 5.8) - this is the first extension point identified for moving Beacon from a demo tool toward a tool analysts could rely on day-to-day.

Feature Data Summary

The two feature spaces differ in size and composition but share the same downstream treatment:

- Network-flow features (348 total): includes duration, packets_count, total_payload_bytes, bytes_rate, packets_rate (all log-transformed, Step 6), delta_start and handshake_duration (structural-missingness candidates, Step 4), and src_port/dst_port (bimodal but not treated as a data issue).

- Memory-dump features (98 total, after dropping info.winBuild which is 100% missing): process-memory structural statistics; no equivalent heavy-tail transform was identified as necessary in the EDA for this feature set.

- Shared label field: category, normalized to exactly 9 values (Backdoor, Benign, Exploit, HackTool, Hoax, Rootkit, Trojan, Virus, Worm) after fixing the "Zbenign"/"Benign" inconsistency.

Data Representation Diagram (Logical ERD)

Since Beacon does not use a relational database, the diagram below represents the logical relationships between its major data entities rather than a physical schema - useful for

understanding how a record moves from raw capture to explained prediction, and as a reference should a persistent store (Section 5.5, D4) be added later.

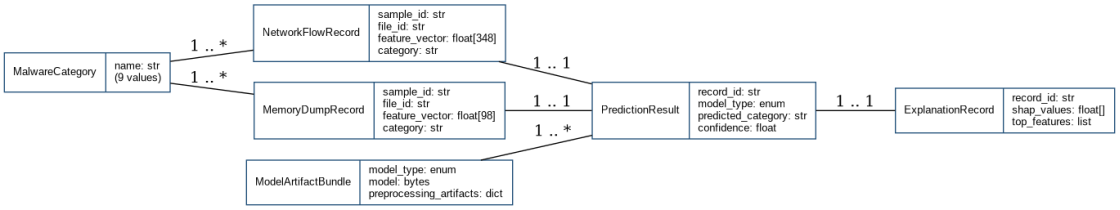

Figure 5.9 - Logical entity-relationship diagram of Beacon's data entities.

5.5.1 Data Dictionary

Beacon follows the object-oriented development approach, so this dictionary lists each object (class), its attributes, and its methods with parameters, in alphabetical order by class name.

ArtifactStore

Attribute	Description
Attribute	— (stateless utility class)
Method (Parameters)	**Description**
save_model(model, path)	Persists a trained classifier to disk via joblib.
load_model(path)	Loads a previously persisted classifier from disk.
save_artifacts(artifacts, path)	Persists a preprocessing artifact bundle (Min-Max ranges, correlation drop-list, feature order, label fixes) to disk.
load_artifacts(path)	Loads a previously persisted preprocessing artifact bundle.

ClusterBasedResampler (extends Resampler)

Attribute	Description
k_range: range	Candidate cluster counts to search (default 2-3, given the small size of the Exploit class).
Method (Parameters)	**Description**
best_k_by_silhouette(X)	Selects the cluster count K that maximizes the Silhouette coefficient for a given feature matrix X.
resample(X, y)	Clusters the minority class, then generates synthetic samples proportionally within each cluster (overrides Resampler.resample).

DashboardController

Attribute	Description
preprocessor: Preprocessor	Reference to the shared preprocessing component.
classifier: MalwareClassifier	Reference to the active Network or Memory classifier.
explainer: Explainer	Reference to the shared SHAP explanation component.
Method (Parameters)	**Description**

Attribute	Description
handle_upload(file)	Reads an uploaded CSV into a DataFrame and validates its schema.
run_pipeline(df)	Calls Preprocessor and the active MalwareClassifier to produce predictions and probabilities for df.
display_results()	Renders predicted categories and confidence scores in the Streamlit dashboard.
display_explanation(row_idx)	Calls Explainer for the specified row and renders the resulting SHAP chart.

Explainer

Attribute	Description
tree_explainer: shap.TreeExplainer	SHAP explainer instance bound to a specific trained model.
Method (Parameters)	**Description**
explain(row)	Returns the SHAP values for a single preprocessed feature row.
top_features(row, n)	Returns the n features with the largest absolute SHAP value for a given row.

MalwareClassifier (abstract; NetworkClassifier and MemoryClassifier are its concrete subclasses)

Attribute	Description
model	The underlying trained XGBoost model object.
feature_order: list	The exact ordered list of feature names the model was trained on, used to align new data at inference.
Method (Parameters)	**Description**
train(X, y)	Fits the underlying model on resampled training data X, y (overridden identically by both subclasses; kept virtual for future model-type flexibility, e.g. a LightGBM subclass).
predict(X)	Returns the predicted category for each row in X.
predict_proba(X)	Returns per-class probability scores for each row in X.
save(path)	Persists the trained model via ArtifactStore.
load(path)	Loads a previously trained model via ArtifactStore.

Preprocessor

Attribute	Description
label_col: str	Name of the target/category column.
log_cols: list	Feature names to log1p-transform (Step 6).
minmax_ranges: dict	Per-feature (min, max) pairs fitted on training data, applied at both training and inference (Step 7).
corr_drop_cols: list	Feature names dropped for exceeding the correlation threshold, fitted on training data (Step 8).
Method (Parameters)	**Description**
remove_duplicates(df)	Drops exact duplicate rows (Step 1).

Attribute	Description
drop_empty_columns(df)	Drops columns that are 100% missing, e.g. info.winBuild (Step 2).
fix_labels(df, label_col)	Normalizes label inconsistencies, e.g. "Zbenign" to "Benign" (Step 3).
handle_missingness(df, label_col, target_class)	Checks whether a column's missingness correlates with a given class label, and chooses sentinel+indicator or median imputation accordingly (Step 4).
log_transform(df, cols)	Applies log1p to the given heavy-tailed columns (Step 6).
fit_minmax(df, cols)	Computes and returns per-column (min, max) ranges from training data (Step 7).
apply_minmax(df, ranges)	Applies previously fitted Min-Max ranges to scale a DataFrame (Step 7).
fit_correlation_prune(df, cols, threshold)	Identifies redundant, highly-correlated columns to drop from training data (Step 8).

Resampler (abstract; SmoteResampler and ClusterBasedResampler are its concrete subclasses)

Attribute	Description
k_neighbors: int	Number of nearest neighbors used for synthetic sample interpolation.
Method (Parameters)	**Description**
resample(X, y)	Returns a class-balanced (X, y) pair; the specific strategy is provided by the subclass.

SmoteResampler (extends Resampler)

Attribute	Description
Attribute	— (inherits k_neighbors from Resampler)
Method (Parameters)	**Description**
resample(X, y)	Applies standard multiclass SMOTE (overrides Resampler.resample); used for the moderately imbalanced Network dataset.

Note: NetworkClassifier and MemoryClassifier are listed together under MalwareClassifier above since they introduce no new attributes or methods beyond overriding train(X, y) with dataset-specific resampling calls - their distinction is behavioral (which Resampler subclass and which trained data they use), not structural.

5.6 User Interface Design

Beacon's Presentation layer (Section 5.2.1) is a single-page dashboard application with four screens, reached through a persistent left-hand sidebar: a landing page ("app"), an Upload Demo screen, a Dashboard screen, and an About screen. The interface is designed for a security analyst who wants to either (a) understand what Beacon does and why, (b) try uploading a sample capture and see how results would be presented, or (c) check the current state of the underlying dataset and pipeline.

From the user's perspective, the flow is: the analyst lands on the landing page, which frames the product ("Behavioral Explainable AI for Cyber Operations Network") and summarizes it at a glance - 9 malware categories, 2 behavioral data streams (network + memory), the XGBoost + SHAP explainability approach, and the scale of the underlying dataset (100GB+ raw behavioral data). From here the analyst can either click "Try the Upload Demo" to go straight to the file-upload screen, or "Learn about the

project" to read the problem statement and technical approach on the About screen. A row of four feature cards (Multiclass Classification, Dual-Source Evidence, Per-Prediction Explanation, Analyst Dashboard) explains what to expect from the rest of the interface before the analyst navigates further.

On the Upload Demo screen, the analyst drops or browses for a CSV of network-flow or memory-forensic features. The interface is explicit about what is and is not functional yet: a caption states that "file parsing below is fully functional; the classification engine itself is still being trained and validated," and an inline message confirms that uploaded files are parsed locally and never leave the analyst's machine. This is a deliberate piece of UI feedback - rather than silently failing or faking a prediction, the interface tells the analyst exactly which part of the system they are previewing.

On the Dashboard screen, the analyst sees the current state of the training data rather than a live model result - again made explicit with an on-screen banner ("REFLECTS DATA PIPELINE STATUS - NOT LIVE MODEL OUTPUT") so this is never mistaken for a real-time detection result. Four summary cards give the total malware categories, network-flow row count, memory-feature row count, and raw dataset volume, followed by a per-category breakdown table showing Network Training Rows alongside Memory Training Rows before and after resampling ("Real" vs. "Final") - directly surfacing the effect of the cluster-based oversampling applied to the Exploit class (Section 5.2.1, Resampling Module) to the analyst, rather than hiding it.

The About screen gives the problem statement and technical approach in plain language ("The Problem", "The Approach") followed by the technology stack the analyst can expect the system to be built on.

Across all four screens, a top-right "Deploy" control and a status banner (e.g. "FRONTEND PREVIEW - MODEL INTEGRATION IN PROGRESS") consistently communicate that this is a frontend preview build, so feedback from analysts or supervisors during review is scoped to the interface and information architecture rather than live model behavior.

5.6.1 Screen Images

The following screens were captured directly from the working frontend build (not a mockup).

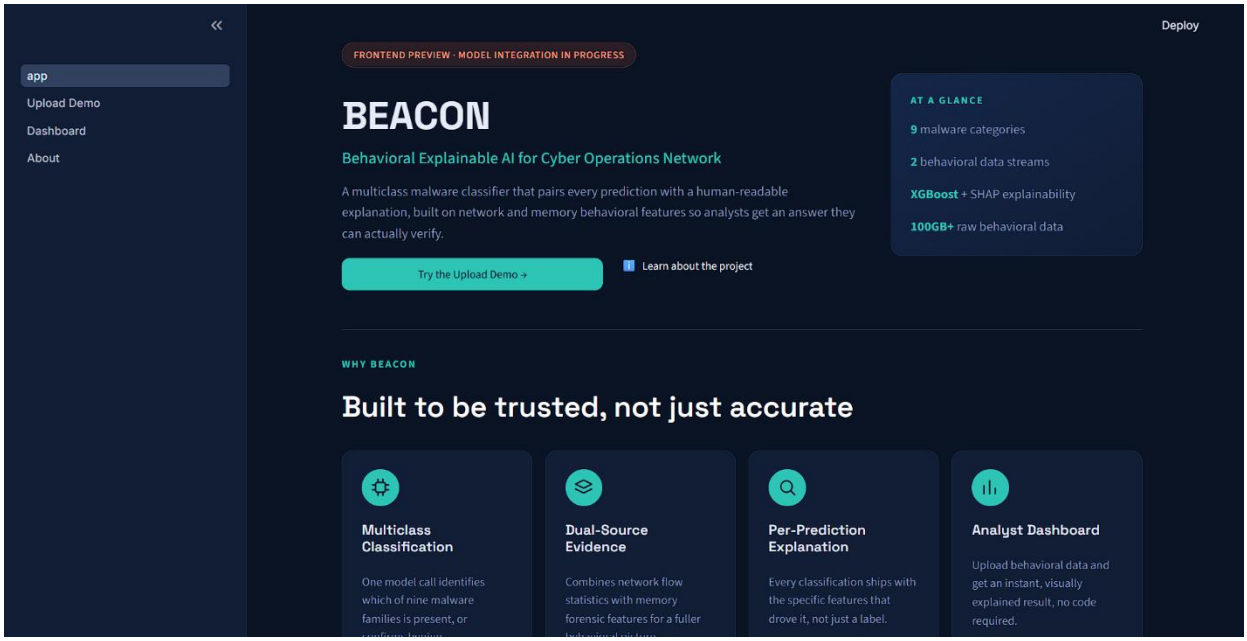

Figure 5.10 - Landing page ("app"), showing the product summary, at-a-glance panel, and "Why Beacon" feature cards.

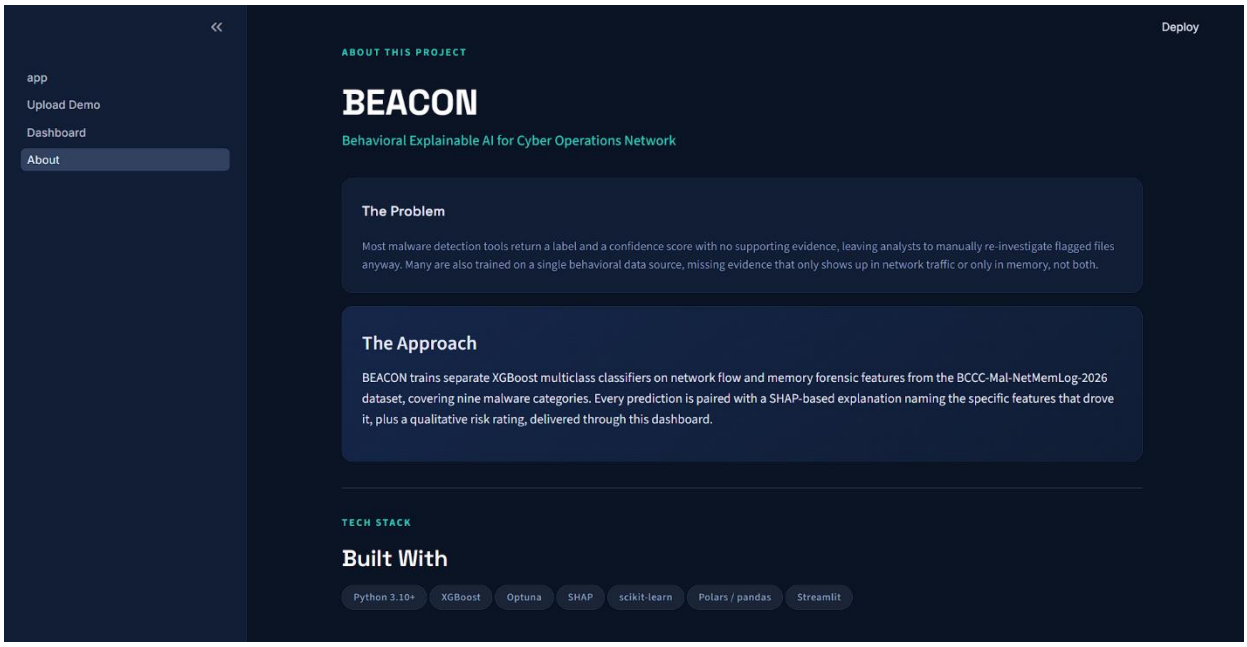

Figure 5.11 - About screen, showing the problem statement, approach, and technology stack.

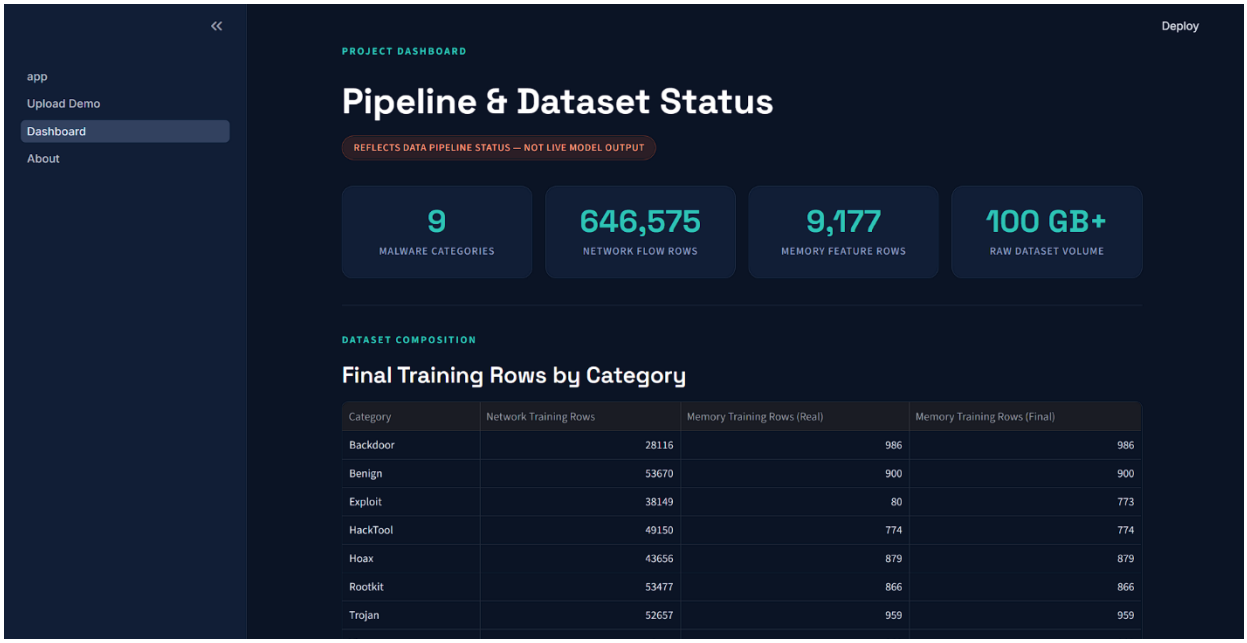

Figure 5.12 - Dashboard screen, showing dataset/pipeline status and the per-category training row breakdown.

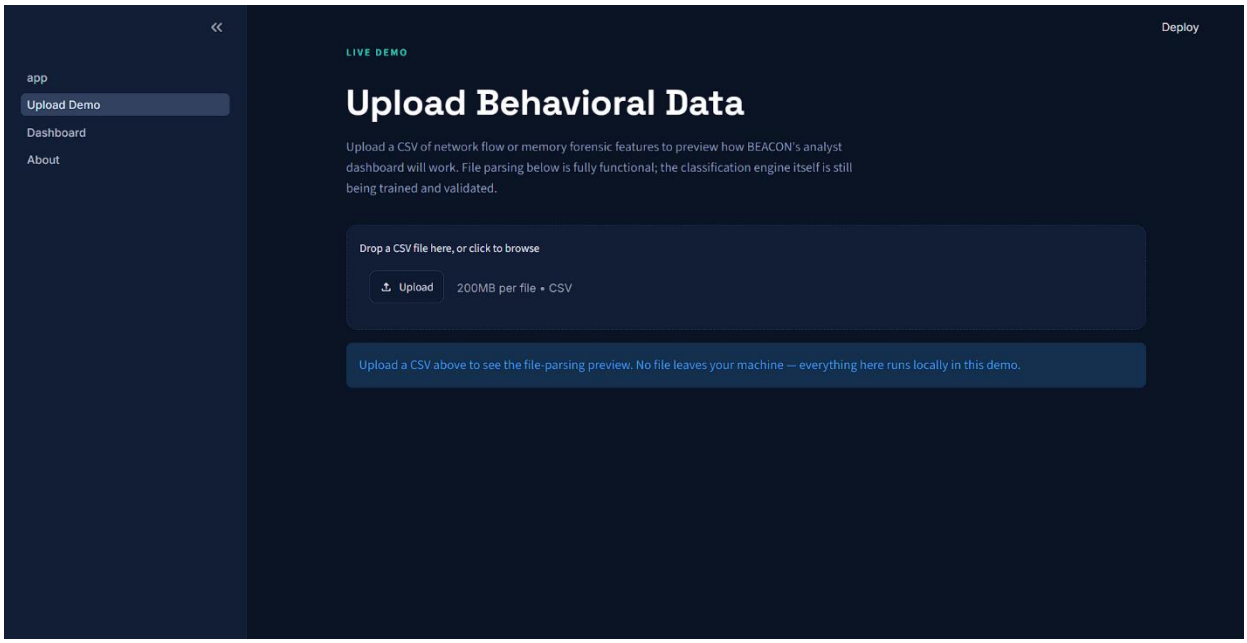

Figure 5.13 - Upload Demo screen, showing the CSV drop zone and local-processing notice.

5.6.2 Screen Objects and Actions

Screen	Object	Action / Feedback
All screens	Sidebar navigation (app, Upload Demo, Dashboard, About)	Click to switch screens; the active screen is highlighted in the sidebar so the analyst always knows their current location.
All screens	"Deploy" control (top-right)	Reserved for publishing/deploying the current build; not part of the analyst-facing detection workflow.
All screens	Status banner (e.g. "FRONTEND PREVIEW - MODEL INTEGRATION IN PROGRESS")	Read-only feedback communicating build status, so screens showing placeholder or pipeline-only data are never mistaken for live model output.
Landing ("app")	"Try the Upload Demo" button	Navigates directly to the Upload Demo screen.
Landing ("app")	"Learn about the project" link	Navigates to the About screen.
Landing ("app")	"At a Glance" panel (9 malware categories, 2 data streams, XGBoost+SHAP, 100GB+ data)	Read-only summary; no action, provides quick context before the analyst commits to exploring further.
Landing ("app")	"Why Beacon" feature cards (Multiclass Classification, Dual-Source Evidence, Per-Prediction Explanation, Analyst Dashboard)	Read-only; each describes one capability the analyst will encounter on the Upload Demo or Dashboard screens.
About	"The Problem" / "The Approach" panels	Read-only explanatory text; no action.
About	"Built With" technology tags (Python, XGBoost, Optuna, SHAP, scikit-	Read-only; communicates the technical stack to a technically literate reviewer (e.g. supervisor or committee).

Screen	Object	Action / Feedback
	learn, Polars/pandas, Streamlit)	
Dashboard	Summary cards (malware categories, network flow rows, memory feature rows, raw dataset volume)	Read-only counts reflecting the current state of the training data, refreshed when the underlying pipeline is re-run.
Dashboard	"Final Training Rows by Category" table	Read-only; scrollable list of all 9 categories with Network Training Rows and Memory Training Rows shown both before ("Real") and after ("Final") resampling, making the imbalance-correction step (Section 5.2.1) visible to the analyst rather than hidden.
Upload Demo	CSV drop zone / "Upload" button	Accepts a dropped or browsed CSV file (network-flow or memory-forensic features, up to 200MB); triggers the file-parsing preview described in Section 5.2.1 (Data Ingestion Module).
Upload Demo	Local-processing notice	Read-only feedback confirming the uploaded file is parsed locally and not transmitted elsewhere during this preview.